Projekt Zaliczeniowy – Bazy Danych

Temat – fitness/siłownia

Autorzy:

Zuzanna Lech

Amelia Skoczylas

Etap 1: Specyfikacja wymagań i diagram związków encji (ERD)

Encja: Klient

Znaczenie encji: Reprezentuje osobę fizyczną, która korzysta z usług siłowni, kupuje karnety i bierze udział w treningach.

Atrybuty:

- **ID_Klienta**
  - **Znaczenie:** Unikalny numer identyfikacyjny przypisywany każdemu klientowi w systemie.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany, klucz główny.
- **Imię**
  - **Znaczenie:** Imię klienta.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Nazwisko**
  - **Znaczenie:** Nazwisko klienta.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **ID_Adresu**
  - **Znaczenie:** Klucz obcy wskazujący na konkretny rekord w encji *Adres_Zamieszkania*, powiązany z danym klientem.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Numer_Telefonu**
  - **Znaczenie:** Numer kontaktowy do szybkiej komunikacji (np. odwołanie zajęć).
  - **Typy atrybutu:** Prosty, wielowartościowy (bo klient może mieć w systemie zapisany np. numer prywatny i służbowy), wymagany.
- **Adres_Email**
  - **Znaczenie:** Adres poczty elektronicznej wykorzystywany do logowania w systemie i przesyłania powiadomień.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.

Encja: Adres_Zamieszkania

Znaczenie encji: Przechowuje szczegółowe dane teleadresowe klientów, co pozwala na zachowanie porządku w danych i łatwiejsze zarządzanie lokalizacjami w systemie.

Atrybuty:

- **ID_Adresu**
  - **Znaczenie:** Unikalny identyfikator adresu w bazie danych, klucz główny.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Ulica**
  - **Znaczenie:** Nazwa ulicy, przy której mieszka klient.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Numer_Domu**
  - **Znaczenie:** Numer budynku oraz lokalu.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Kod_Pocztowy**
  - **Znaczenie:** Sześciocyfrowy kod pocztowy (np. 00-001).
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Miasto**
  - **Znaczenie:** Nazwa miejscowości.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.

Encja: Pracownik

Znaczenie encji: Reprezentuje osobę zatrudnioną w klubie fitness, która odpowiada za obsługę klientów, prowadzenie zajęć, utrzymanie czystości lub zarządzanie obiektem.

Atrybuty:

- **ID_Pracownika**
  - **Znaczenie:** Unikalny numer identyfikacyjny przypisywany pracownikowi w systemie, klucz główny.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Imię**
  - **Znaczenie:** Imię pracownika.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Nazwisko**
  - **Znaczenie:** Nazwisko pracownika.
  - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Stanowisko**
  - **Znaczenie:** Rola i funkcja, jaką pracownik pełni w klubie (np. trener personalny, recepcjonista, menedżer, osoba sprzątająca).

    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Numer_Telefonu**
    - **Znaczenie:** Numer kontaktowy do pracownika.
    - **Typy atrybutu:** Prosty, wielowartościowy (pracownik może podać numer prywatny i służbowy), wymagany.
- **Data_Zatrudnienia**
    - **Znaczenie:** Data rozpoczęcia pracy w klubie fitness.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Wynagrodzenie**
    - **Znaczenie:** Podstawowa pensja pracownika (np. stawka godzinowa lub miesięczna).
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.

Encja: Karnet

Znaczenie encji: Reprezentuje ofertę abonamentową lub rodzaj wejściówki, którą klient może wykupić, aby uzyskać dostęp do usług i stref klubu fitness.

Atrybuty:

- **ID_Karnetu**
    - **Znaczenie:** Unikalny numer identyfikacyjny przypisywany każdemu rodzajowi karnetu w ofercie.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany, klucz główny.
- **ID_Klienta**
    - **Znaczenie:** Klucz obcy wskazujący, który klient zakupił dany karnet i jest jego właścicielem.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Nazwa**
    - **Znaczenie:** Handlowa nazwa karnetu (np. "Open", "Poranny", "Student", "Vip", “Jednorazowy”).
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Cena**
    - **Znaczenie:** Koszt zakupu danego karnetu (np. miesięczny lub jednorazowy).
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Czas_Trwania_Dni**

    - **Znaczenie:** Okres, na jaki wydawany jest karnet, wyrażony w liczbie dni (np. 30, 90, 365).
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Data_Rozpoczęcia**
    - **Znaczenie:** Data, od której dany karnet staje się aktywny i uprawnia klienta do korzystania z usług klubu.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Data_Zakończenia**
    - **Znaczenie:** Data wygaśnięcia ważności karnetu, po której dostęp do usług zostaje zablokowany.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Dostępne_Strefy**
    - **Znaczenie:** Określa, z jakich udogodnień w ramach klubu można korzystać (np. "Siłownia", "Basen", "Strefa Cardio", "Sauna").
    - **Typy atrybutu:** Prosty, wielowartościowy (karnet może uprawniać do wejścia do kilku stref naraz), wymagany.

Encja: Zajęcia_Grupowe

Znaczenie encji: Reprezentuje zaplanowane jednostki treningowe prowadzone przez trenera dla grupy klientów (np. Joga, CrossFit, Spinning).

Atrybuty:

- **ID_Zajęć**
    - **Znaczenie:** Unikalny identyfikator konkretnej sesji treningowej, klucz główny.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Nazwa**
    - **Znaczenie:** Tytuł zajęć określający ich rodzaj.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Termin**
    - **Znaczenie:** Czas rozpoczęcia i zakończenia treningu.
    - **Typy atrybutu:** Złożony (składa się z Daty, Godziny_Rozpoczęcia i Godziny_Zakończenia), jednowartościowy, wymagany.
- **Limit_Miejsc**
    - **Znaczenie:** Maksymalna liczba uczestników, którzy mogą zapisać się na zajęcia.

    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Poziom_Trudności**
    - **Znaczenie:** Wskazówka dla klientów (np. początkujący, średniozaawansowany, zaawansowany).
    - **Typy atrybutu:** Prosty, jednowartościowy, opcjonalny.
- **ID_Sali**
    - **Znaczenie:** Klucz obcy wskazujący, w którym pomieszczeniu odbywa się trening.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **ID_Pracownika**
    - **Znaczenie:** Klucz obcy wskazujący na konkretnego pracownika (trenera), który jest przypisany do poprowadzenia tych zajęć.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.

Encja: Rezerwacje

Znaczenie encji: Reprezentuje fakt zapisania się konkretnego klienta na konkretne zajęcia grupowe. Pełni rolę tzw. encji asocjacyjnej, która rozbija problematyczną relację wiele do wielu (N:M) między klientami a zajęciami na dwie bezpieczne relacje jeden do wielu (1:N).

Atrybuty:

- **ID_Rezerwacji**
    - **Znaczenie:** Unikalny numer identyfikacyjny konkretnego wpisu o rezerwacji, klucz główny.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **ID_Klienta**
    - **Znaczenie:** Wskazuje, który klient dokonał zapisu (to klucz obcy powiązany z encją *Klient*).
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **ID_Zajęć**
    - **Znaczenie:** Wskazuje, na które zajęcia zapisał się klient (to klucz obcy powiązany z encją *Zajęcia_Grupowe*).
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Data_Złożenia_Rezerwacji**
    - **Znaczenie:** Dokładna data i czas w systemie, kiedy klient dokonał rezerwacji zajęć.

    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Status**
    - **Znaczenie:** Aktualny stan rezerwacji, np. "Aktywna", "Anulowana przez klienta", "Obecny".
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany

Encja: Sala

Znaczenie encji: Reprezentuje fizyczną przestrzeń w klubie fitness przeznaczoną do ćwiczeń lub prowadzenia zajęć.

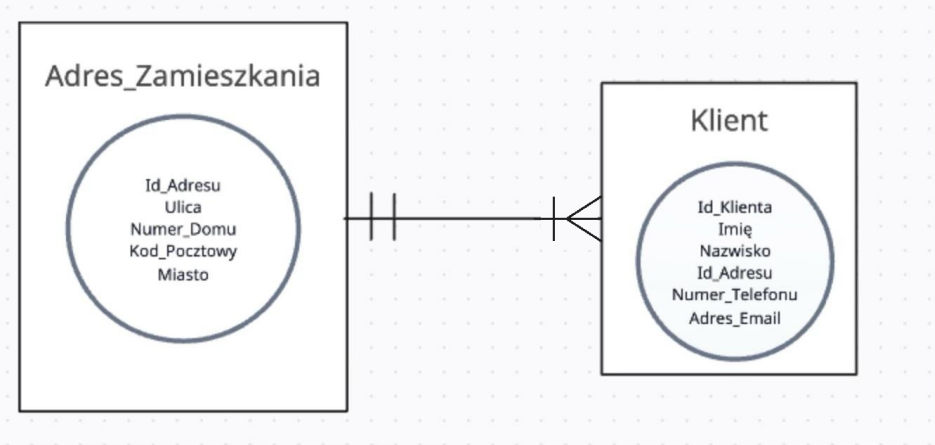
Atrybuty:

- **ID_Sali**
    - **Znaczenie:** Unikalny numer identyfikacyjny pomieszczenia.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Nazwa**
    - **Znaczenie:** Nazwa własna lub numer sali (np. "Studio A", "Sala Rowery").
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.
- **Pojemność**
    - **Znaczenie:** Maksymalna liczba osób mogących bezpiecznie przebywać w sali.
    - **Typy atrybutu:** Prosty, jednowartościowy, wymagany.

Znaczenie związków między encjami (Relacje):

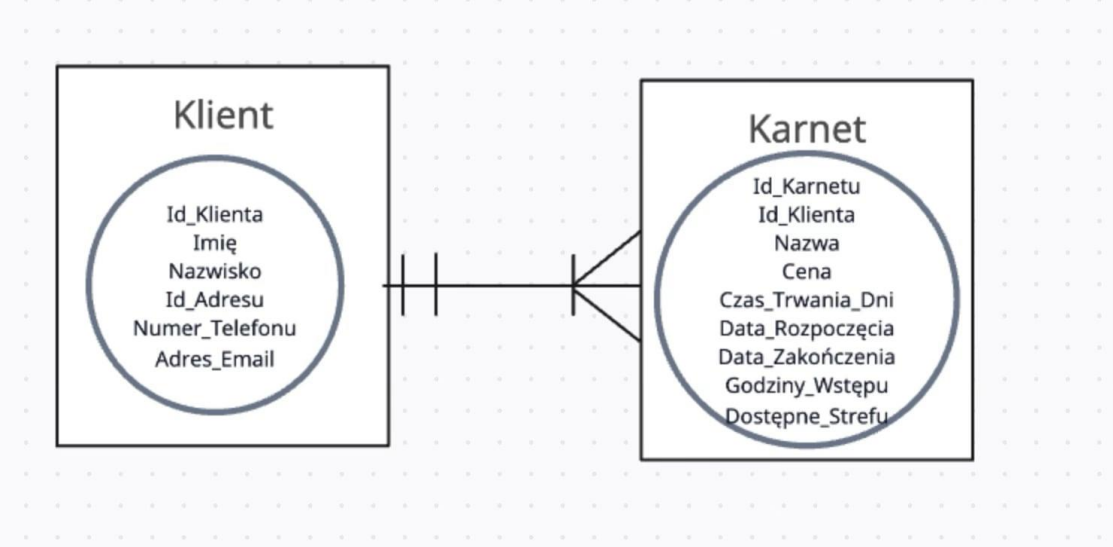
Opis relacji: Adres_Zamieszkania – Klient

- **Rodzaj relacji:** 1:N (Jeden do Wielu).
- **Znaczenie związku:** Jeden adres w bazie może być przypisany do wielu klientów (np. członków jednego gospodarstwa domowego), natomiast każdy klient ma dokładnie jeden adres zamieszkania. W danych testowych klienci nr 1 i 9 korzystają ze wspólnego wpisu adresowego (`Id_Adresu` = 1).
- **Implementacja:** Relacja ta w modelu fizycznym jest realizowana poprzez umieszczenie klucza obcego `Id_Adresu` w encji Klient. Na kolumnie nie założono ograniczenia UNIQUE, dzięki czemu jeden adres może być współdzielony.



Opis relacji: Klient – Karnet

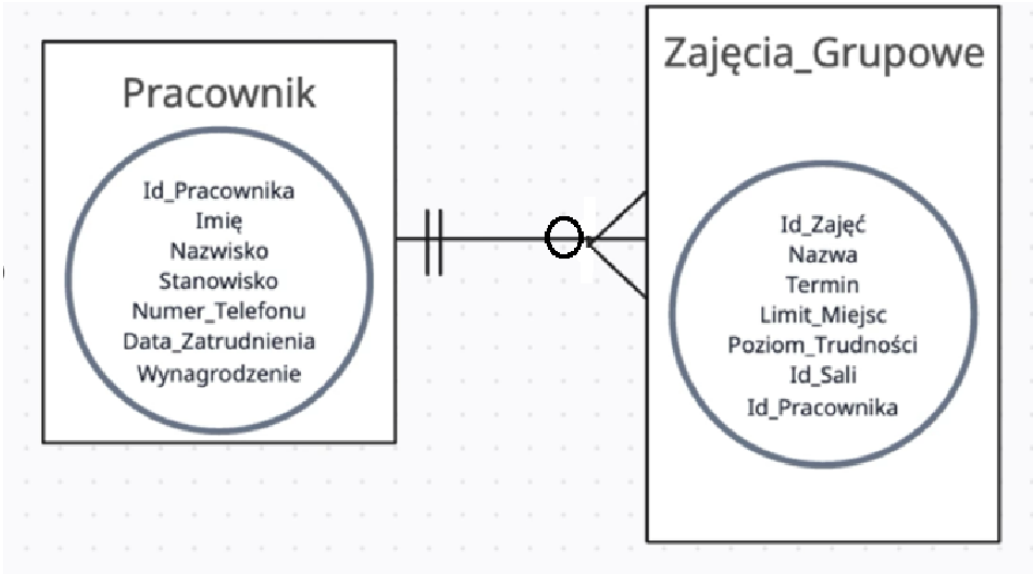
- **Rodzaj relacji:** 1:N (Jeden do Wielu).
- **Znaczenie związku:** Jeden Klient może w historii swojej aktywności zakupić wiele (N) Karnetów (np. jeden po drugim po wygaśnięciu poprzedniego). Jednak jeden konkretny Karnet o danym numerze ID jest przypisany wyłącznie do jednego Klienta.
- **Implementacja:** Relacja jest realizowana przez klucz obcy ID_Klienta umieszczony w encji Karnet.



Opis relacji: Pracownik – Zajęcia_Grupowe

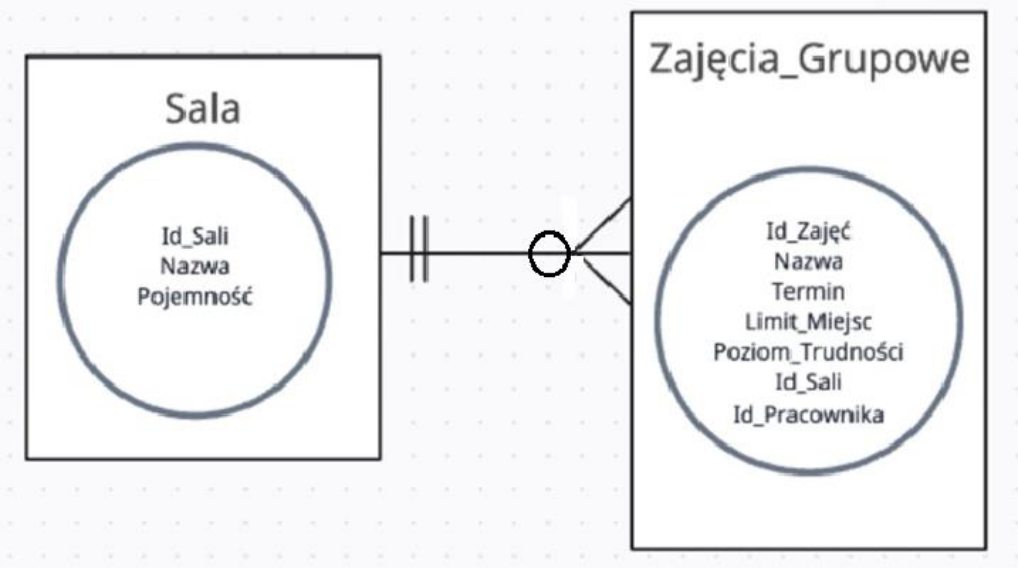
- **Rodzaj relacji:** 1:N (Jeden do Wielu).

- **Znaczenie związku:** Jeden Pracownik (trener) może(ale nie musi) prowadzić wiele (N) różnych Zajęć_Grupowych w różnych terminach w ramach swojego grafiku pracy. Z kolei każde pojedyncze, konkretne Zajęcia_Grupowe są prowadzone i przypisane do dokładnie jednego Pracownika.
- **Implementacja:** Relacja ta w modelu fizycznym jest realizowana poprzez dodanie klucza obcego ID_Pracownika w tabeli Zajęcia_Grupowe.



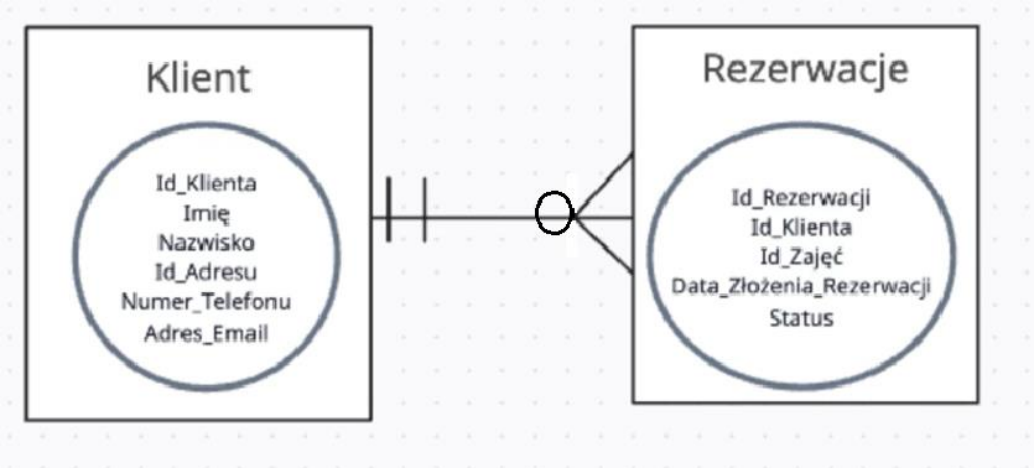
Opis relacji: Sala – Zajęcia_Grupowe

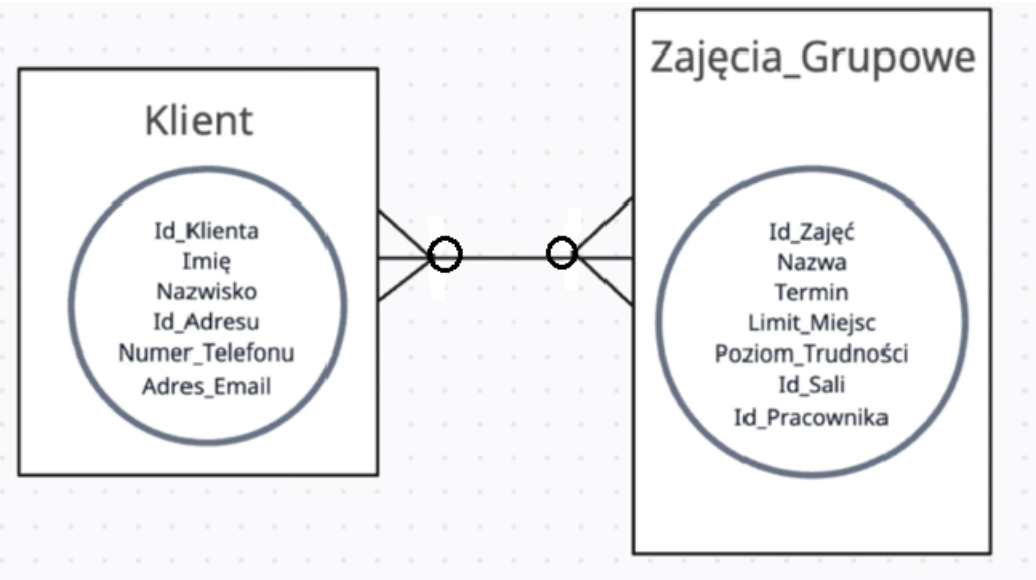
- **Rodzaj relacji:** 1:N (Jeden do Wielu).
- **Znaczenie związku:** W jednej, konkretnej Sali może (ale nie musi) odbywać się wiele (N) różnych Zajęć_Grupowych w różnych godzinach i dniach (np. rano Joga, po południu Pilates). Jednocześnie każde konkretne Zajęcia_Grupowe (dana sesja z grafiku) są przypisane wyłącznie do jednej konkretnej Sali.
- **Implementacja:** W modelu fizycznym relacja ta jest realizowana poprzez umieszczenie klucza obcego ID_Sali w tabeli Zajęcia_Grupowe.

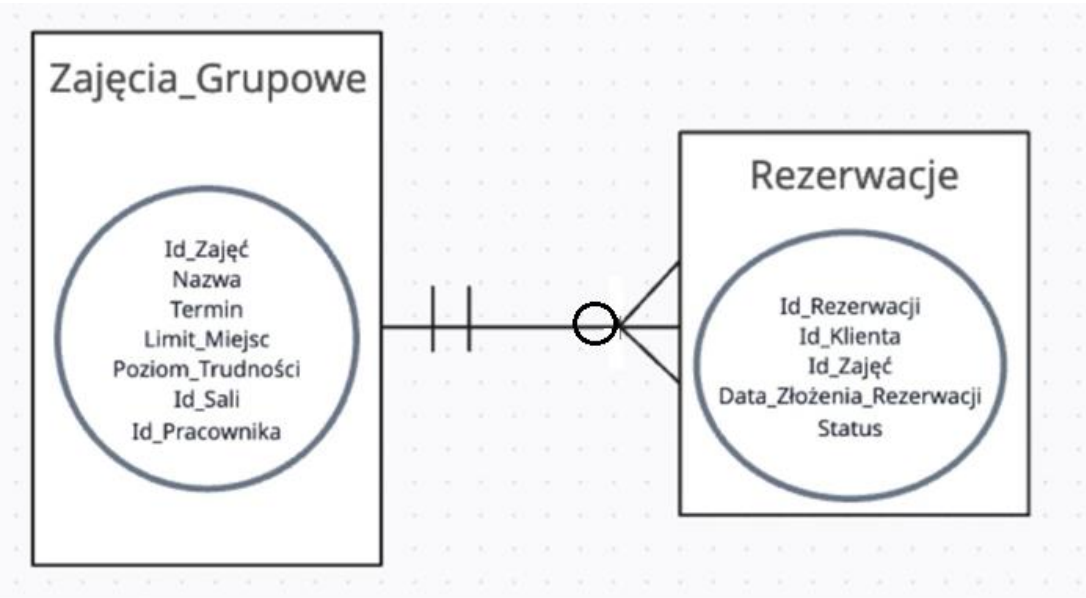


Opis relacji: Klient – Zajęcia_Grupowe:

- **Rodzaj relacji:** wiele do wielu (N:M)
- **Klient – Rezerwacje (Relacja 1:N):** Jeden klient może złożyć wiele rezerwacji (lub może nie mieć żadnej), ale jedna konkretna rezerwacja należy tylko do jednego klienta.
- **Zajęcia_Grupowe – Rezerwacje (Relacja 1:N):** Na jedne zajęcia składa się wiele rezerwacji (wielu klientów), ale pojedynczy wpis rezerwacji dotyczy tylko jednych, konkretnych zajęć.
- **Klient – Zajęcia_Grupowe (Relacja N:M zrealizowana przez Rezerwacje):** Wielu klientów uczestniczy w wielu zajęciach. Relacja ta na diagramie ERD jest poprowadzona właśnie przez encję łączącą Rezerwacje.

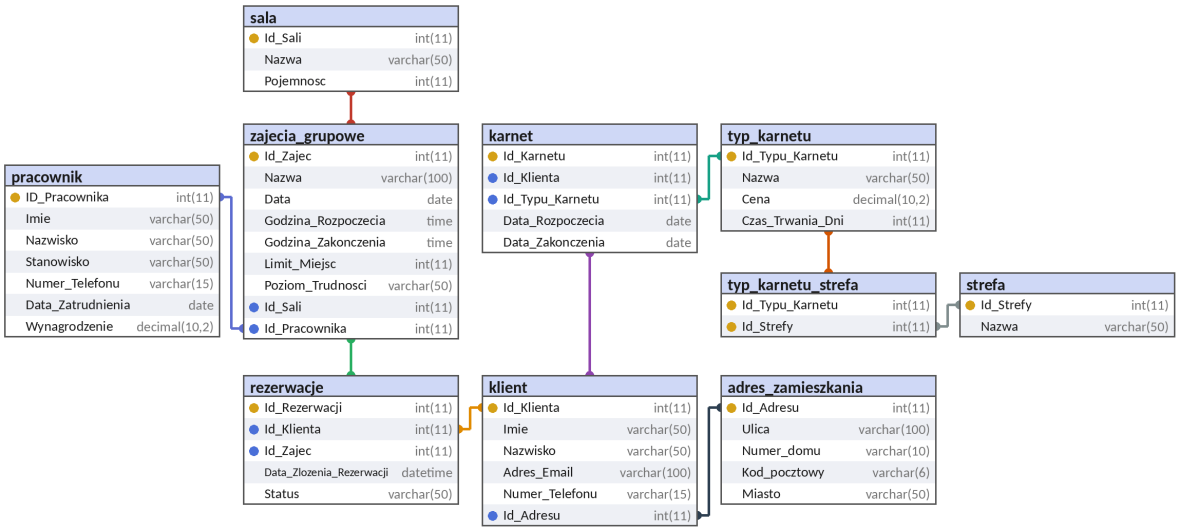






Etap 2: Projekt, implementacja i testy bazy danych

2. Opracowanie diagramu fizycznego modelu bazy danych zgodnego z 3. postacią normalną:



Zgodność z postaciami normalnymi:

- **Pierwsza postać normalna:** Każda tabela posiada zdefiniowany klucz główny typu `INT AUTO_INCREMENT` (w tabeli łączącej `Typ_Karnetu_Strefa` – klucz złożony). Wszystkie kolumny przechowują wartości atomowe (niepodzielne). Przykładem zastosowania 1NF było rozbicie złożonego atrybutu „Termin" na osobne kolumny `Data`, `Godzina_Rozpoczecia` i `Godzina_Zakonczenia` w tabeli `Zajecia_Grupowe` oraz zamiana wielowartościowego atrybutu „Dostępne_Strefy" na tabele `Strefa` i `Typ_Karnetu_Strefa`. W tabelach `Klient` i `Pracownik` przechowywany jest jeden (atomowy) numer telefonu.
- **Druga postać normalna:** Model automatycznie spełnia ten warunek, ponieważ każda z tabel opiera się na prostym, pojedynczym kluczu głównym (sztucznym, np. `Id_Klienta`, `Id_Karnetu`); tabela łącząca `Typ_Karnetu_Strefa` nie ma atrybutów spoza klucza. Żadna kolumna nie jest zależna tylko od części klucza głównego.
- **Trzecia postać normalna:** Wyeliminowano zależności przechodnie (zależności między atrybutami niebędącymi kluczem). Dane adresowe wydzielono do tabeli `Adres_Zamieszkania`, a dane rodzaju karnetu (`Nazwa`, `Cena`, `Czas_Trwania_Dni`) do tabeli `Typ_Karnetu`. Tabela `Karnet` zapisuje już tylko fakt zakupu (klient, rodzaj, daty), więc każdy jej atrybut zależy wyłącznie od `Id_Karnetu`.

3. Implementacja bazy danych

Projekt został zaimplementowany w systemie zarządzania bazą danych przy użyciu języka SQL. Podczas tworzenia tabel (instrukcje `CREATE TABLE`) wykorzystano odpowiednie typy danych (m.in. `INT`, `VARCHAR`, `DECIMAL`, `DATE`, `TIME`, `DATETIME`).

Każda tabela posiada zdefiniowany klucz główny (`PRIMARY KEY`) z atrybutem `AUTO_INCREMENT`. Integracja danych i relacje zostały zapewnione przez klucze obce (`FOREIGN KEY REFERENCES`). Określono również wymagalność danych (`NOT NULL` lub `NULL` dla kolumn opcjonalnych). Ograniczenie unikalności adresu e-mail klienta (`UNIQUE`) nie zostało jeszcze nałożone – zaplanowano je w README (sekcja „Do poprawy”). Ponadto, w celu weryfikacji poprawności struktury, do każdej z tabel wprowadzono przykładowe dane testowe za pomocą instrukcji `INSERT INTO`.

4. Testowanie i walidacja:

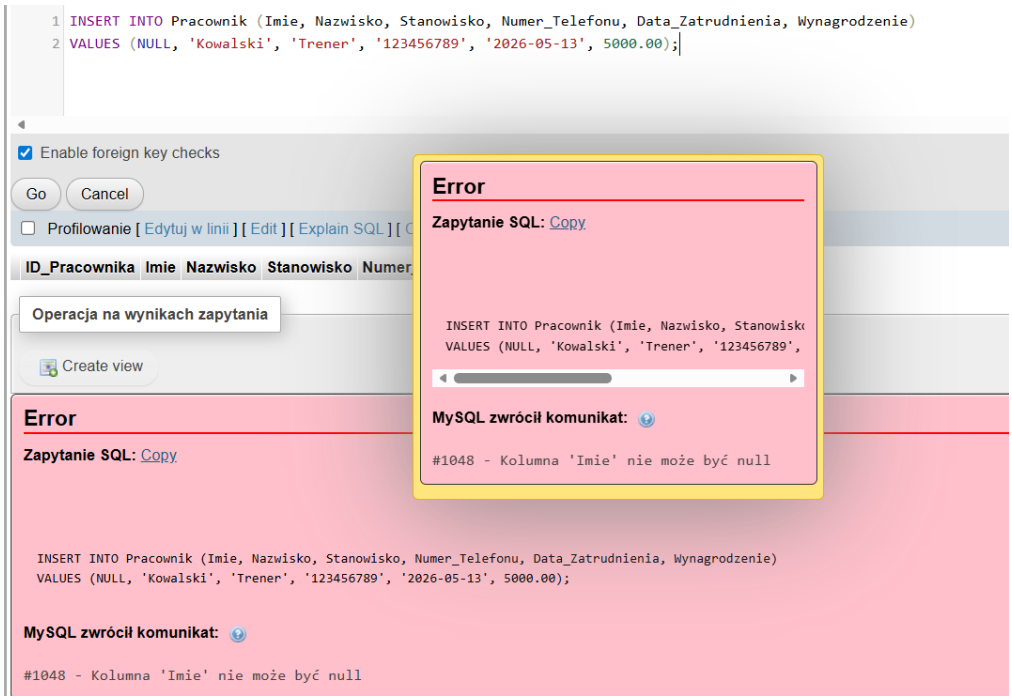
Baza danych została poddana testom mającym na celu weryfikację poprawności wdrożonych ograniczeń i zabezpieczeń. System relacyjny blokuje próby naruszenia spójności (`NOT NULL`, klucze), z wyjątkiem niepoprawnych wartości liczbowych (Test 2).

4.1. Integralność semantyczna

System weryfikuje wymagalność danych (`NOT NULL`) oraz ograniczenia długości znaków. Zgodność z typami danych jest sprawdzana tylko częściowo – bez trybu strict tekst wpisany do pola liczbowego zostaje po cichu zamieniony na 0 (patrz Test 2).

Test 1: Weryfikacja wymagalności danych (NOT NULL)

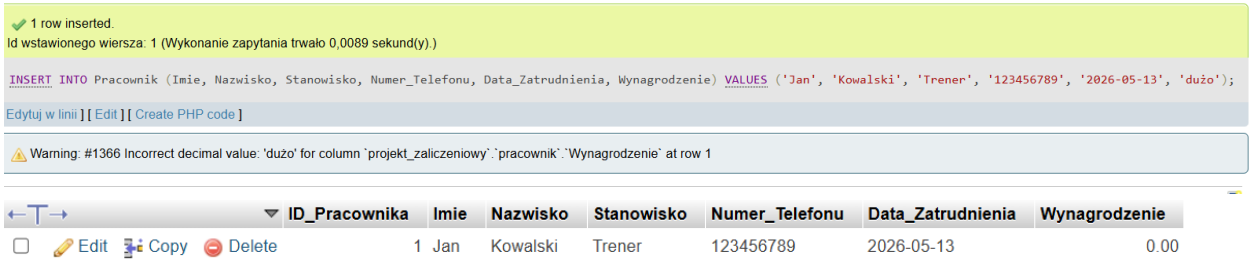
- **Scenariusz testowy:** Próba dodania pracownika bez podania obowiązkowego imienia (wstawienie wartości NULL w pole Imie).



- **Wynik testu (Sukces):** Baza odrzuca zapytanie. Wyświetla błąd dotyczący naruszenia ograniczenia NOT NULL dla kolumny Imie.

Test 2: Weryfikacja poprawności typów danych

- **Scenariusz testowy:** Próba dodania pracownika z tekstem zamiast wartości liczbowej w polu Wynagrodzenie (zdefiniowanym jako typ DECIMAL).

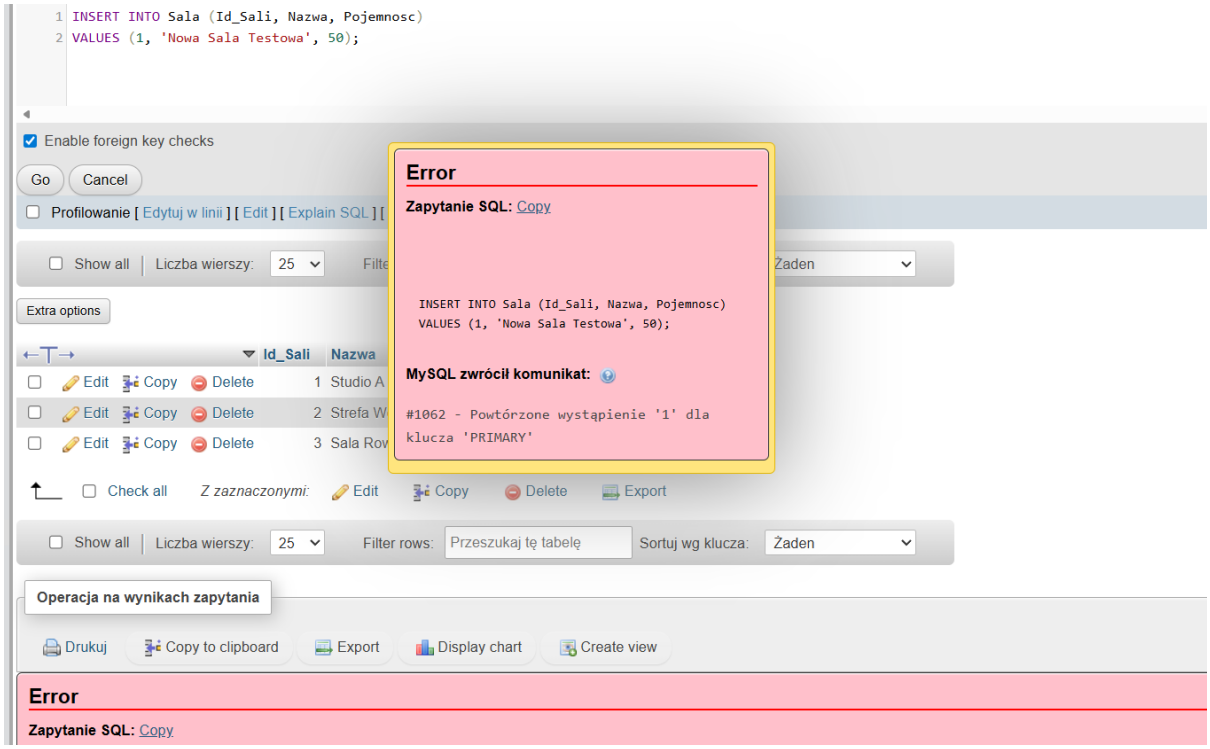


- **Wynik testu (wykryta słabość):** Zapytanie nie zostało odrzucone. Serwer zgłosił jedynie ostrzeżenie (Warning: #1366 Incorrect decimal value) i zapisał w polu Wynagrodzenie wartość 0.00 zamiast tekstu „dużo” – wiersz został dodany do tabeli (ID 1). Dzieje się tak, ponieważ serwer działa w trybie bez ścisłej kontroli (SQL mode bez `STRICT_TRANS_TABLES`), więc niepoprawne wartości liczbowe są po cichu zamieniane na wartość domyślną. Po włączeniu trybu strict (np. `SET sql_mode = 'STRICT_TRANS_TABLES';`) ten sam `INSERT` zostałby odrzucony z błędem #1366 (patrz README, sekcja „Do poprawy”).

4.2. Integralność encji

Baza danych dba o to, aby każdy wiersz w tabeli był unikalny i możliwy do zidentyfikowania, dzięki ograniczeniu `PRIMARY KEY` (które wymusza unikalność i brak wartości pustych).

- **Scenariusz testowy:** Próba dodania nowej Sali z ręcznie wpisanym numerem `Id_Sali`, który już istnieje w bazie (np. 1).

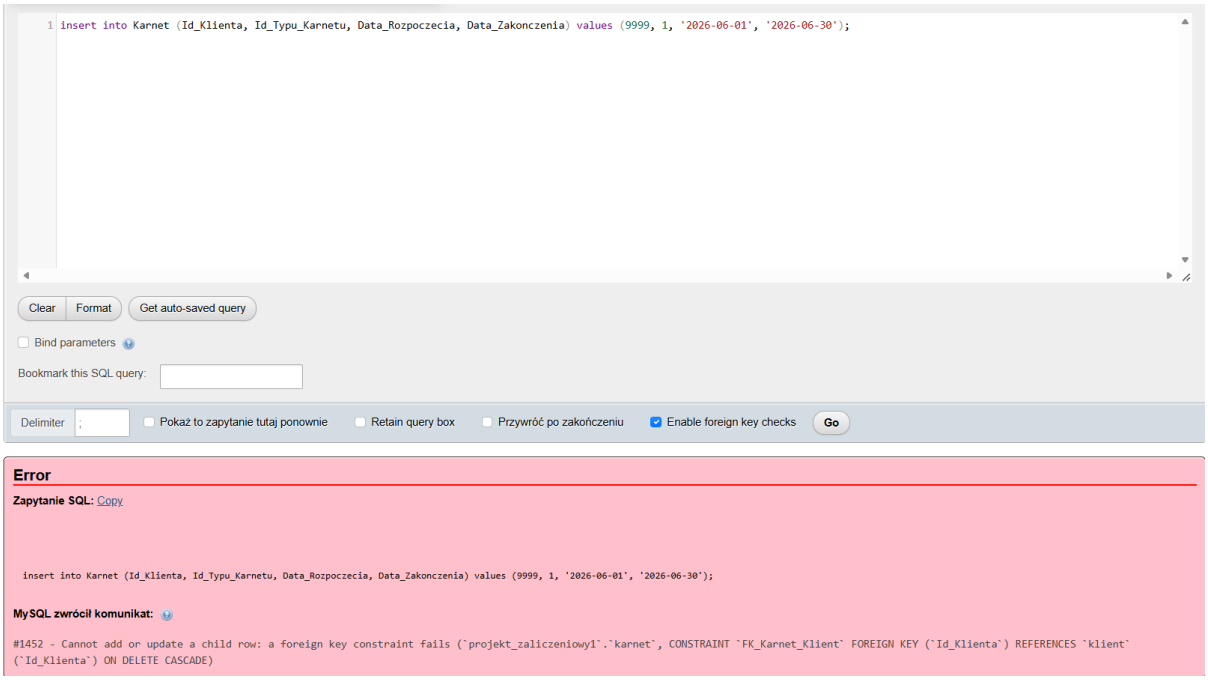


- **Wynik testu (Sukces):** Baza odrzuca zapytanie ze względu na duplikację klucza głównego (Error: Powtórzono wystąpienie ‘1’ dla klucza ‘PRIMARY’).

4.3. Integralność referencyjna

Sprawdzono poprawność działania kluczy obcych (`FOREIGN KEY`). Baza nie pozwala na przypisanie danych do nieistniejących rekordów nadrzędnych.

- **Scenariusz testowy:** Próba przypisania nowego karnetu do klienta, który nie istnieje w systemie (np. o wymyślonym `Id_Klienta` = 9999).



- **Wynik testu (Sukces):** Baza odrzuca zapytanie, chroniąc przed powstaniem osieroconego rekordu. Wyświetla błąd naruszenia klucza obcego (Error: Cannot add or update a child row: a foreign key constraint fails)

Etap 3: Raporty i widoki

5. Utworzenie co najmniej 4 raportów w postaci widoków SQL

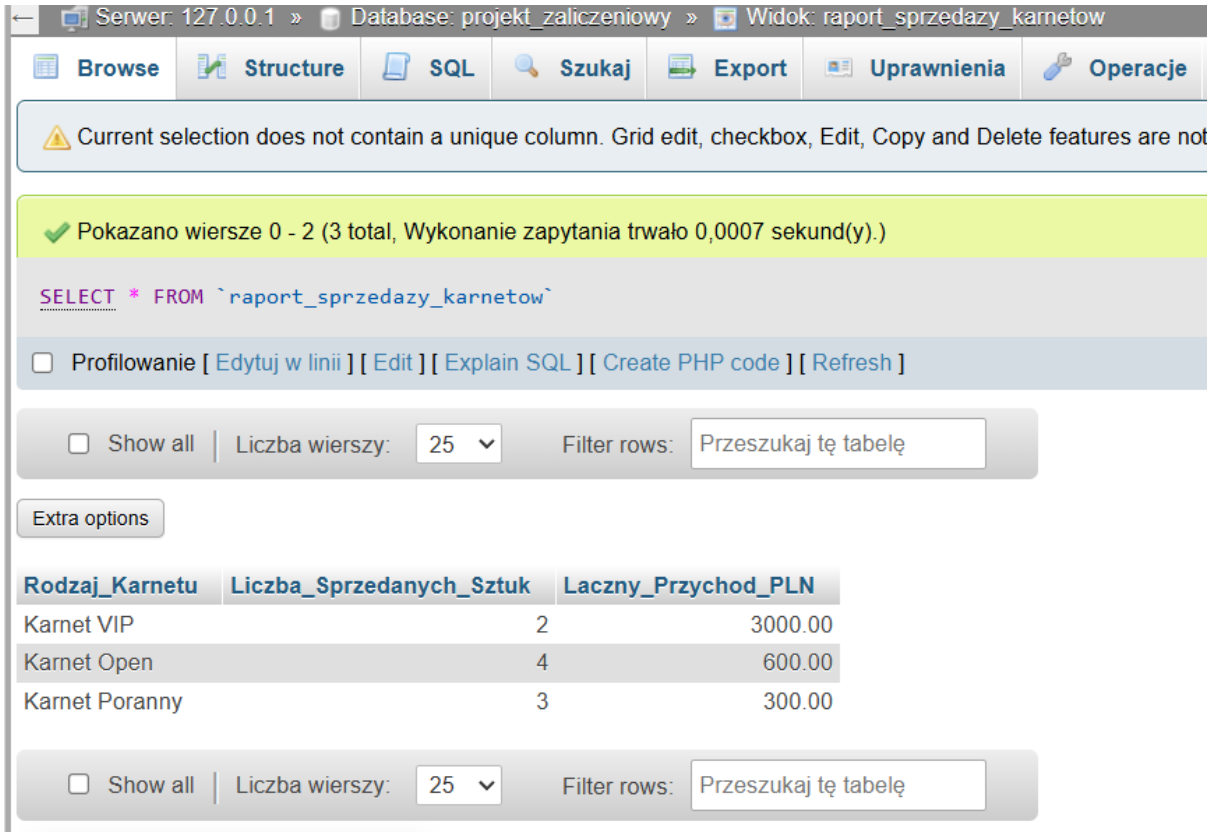
Aby ułatwić zarządzanie klubem fitness oraz wyciąganie kluczowych statystyk biznesowych, w bazie utworzono 5 raportów w postaci widoków. Spełniają one zasady dobrych praktyk, posiadają czytelne aliasy oraz wykorzystują zaawansowane mechanizmy języka SQL (łączenia JOIN, agregacje, podzapytania).

Raport 1: Podsumowanie sprzedaży karnetów

- **Praktyczne zastosowanie:** Widok pokazuje, które typy karnetów sprzedają się najlepiej i jaki generują łączny przychód.
- **Wykorzystane mechanizmy:** Funkcje agregujące (`SUM`, `COUNT`), grupowanie (`GROUP BY`), sortowanie (`ORDER BY DESC`).

SQL

```
CREATE VIEW Raport_Sprzedazy_Karnetow AS
SELECT
    Nazwa AS Rodzaj_Karnetu,
    COUNT(Id_Karnetu) AS Liczba_Sprzedanych_Sztuk,
    SUM(Cena) AS Laczny_Przychod_PLN
FROM Karnet
JOIN Typ_Karnetu USING (Id_Typu_Karnetu)
GROUP BY Nazwa
ORDER BY Laczny_Przychod_PLN DESC;
```

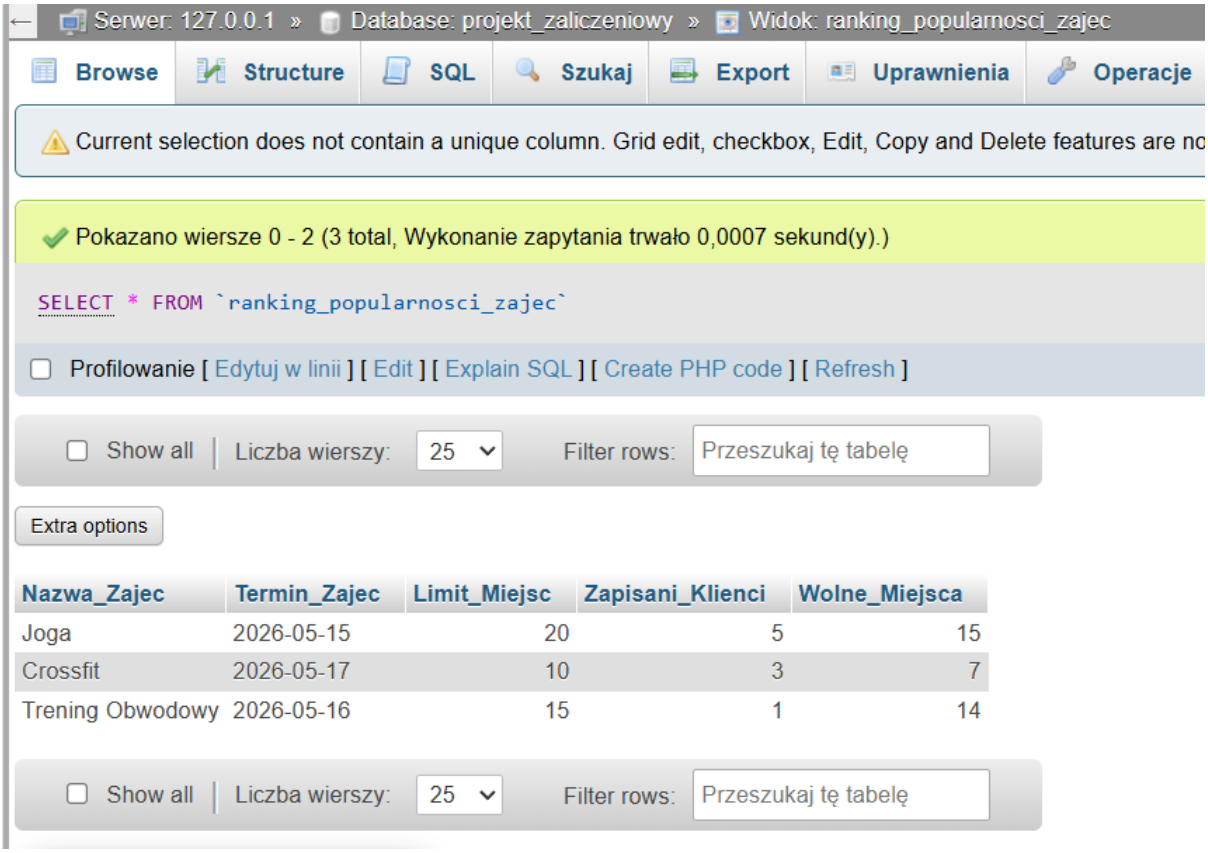


Raport 2: Frekwencja na zajęciach grupowych (Ranking popularności)

- **Praktyczne zastosowanie:** Menadżer klubu widzi, ile osób zapisało się na konkretne zajęcia i ile zostało wolnych miejsc.
- **Wykorzystane mechanizmy:** Łączenie tabel (`LEFT JOIN`), funkcje agregujące (`COUNT`), operacje matematyczne (obliczanie wolnych miejsc), grupowanie.

SQL

```
CREATE VIEW Ranking_Popularnosci_Zajec AS
SELECT
    zg.Nazwa AS Nazwa_Zajec,
    zg.Data AS Termin_Zajec,
    zg.Limit_Miejsc,
    COUNT(r.Id_Rezerwacji) AS Zapisani_Klienci,
    (zg.Limit_Miejsc - COUNT(r.Id_Rezerwacji)) AS Wolne_Miejsca
FROM Zajecia_Grupowe zg
LEFT JOIN Rezerwacje r ON zg.Id_Zajec = r.Id_Zajec AND r.Status =
'Aktywna'
GROUP BY zg.Id_Zajec, zg.Nazwa, zg.Data, zg.Limit_Miejsc
ORDER BY Zapisani_Klienci DESC;
```

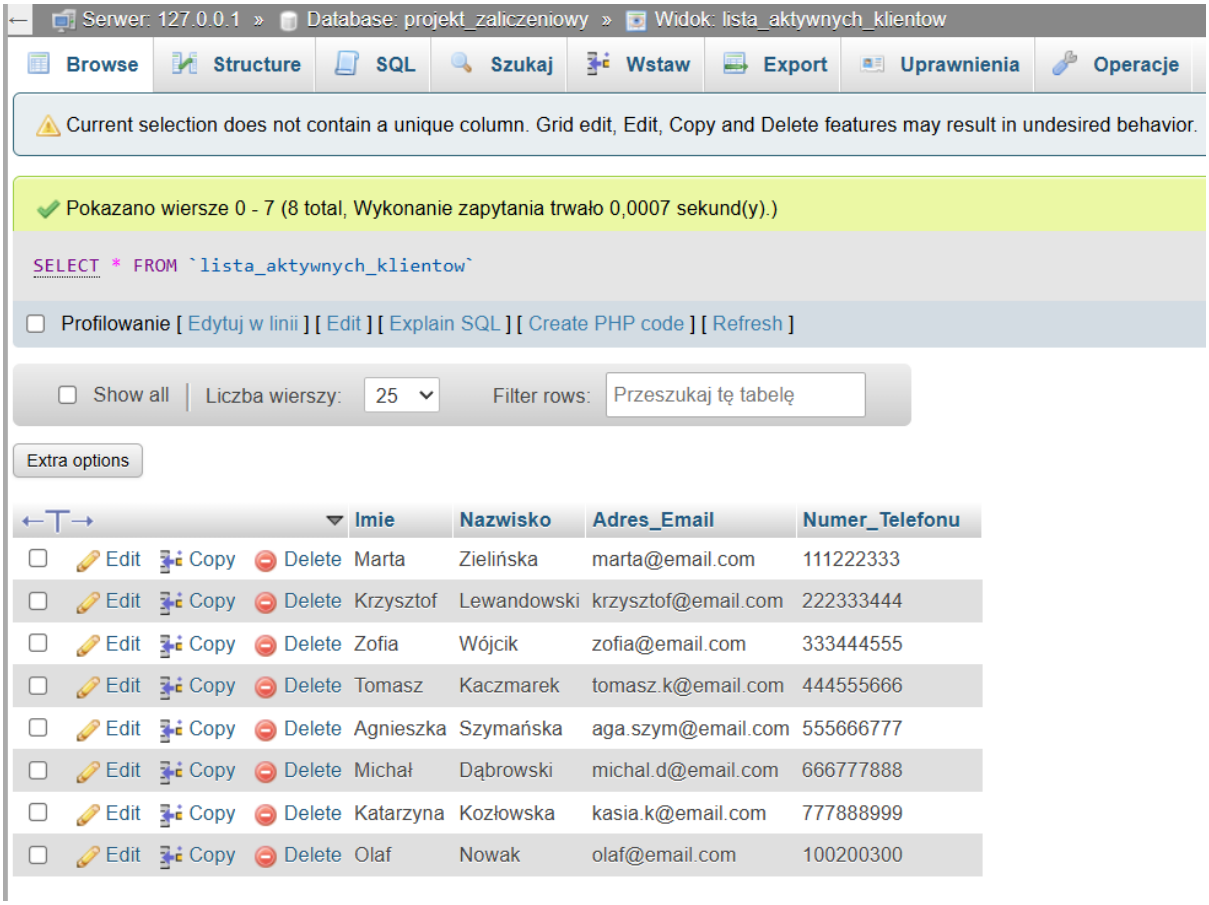


Raport 3: Klienci z aktywnym karnetem

- **Praktyczne zastosowanie:** Szybka lista mailingowa lub kontaktowa do aktywnych klientów klubu (np. w celu wysłania newslettera).
- **Wykorzystane mechanizmy:** Podzapytanie w klauzuli warunkowej (`WHERE ... IN (SELECT ...)`), weryfikacja daty (`CURDATE()`).

SQL

```
CREATE VIEW Lista_Aktywnych_Klientow AS
SELECT
    Imie,
    Nazwisko,
    Adres_Email,
    Numer_Telefonu
FROM Klient
WHERE Id_Klienta IN (
    SELECT Id_Klienta
    FROM Karnet
    WHERE Data_Zakonczenia >= CURDATE()
);
```



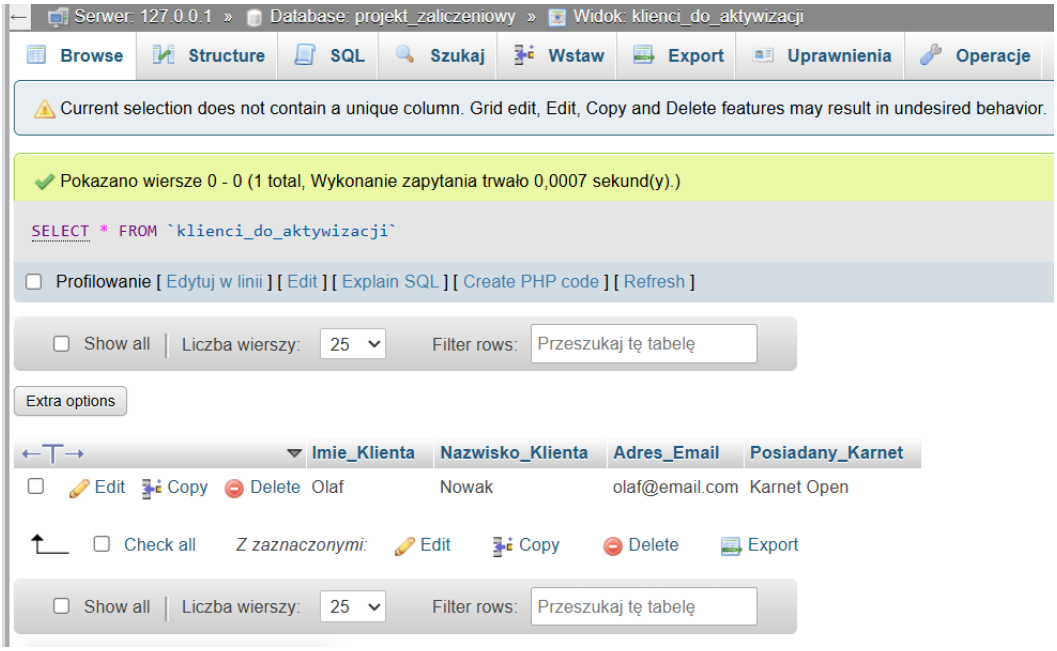
Raport 4: Klienci z aktywnym karnetem, ale bez żadnych rezerwacji (EXISTS / NOT EXISTS)

- **Praktyczne zastosowanie:** Pokazuje osoby, które zapłaciły za karnet, ale nie zapisały się na żadne zajęcia. Idealne zestawienie dla działu marketingu. Można do nich wysłać maila motywacyjnego lub zaoferować darmową konsultację, by ich zaktywizować i zapobiec rezygnacji z usług klubu.

- **Wykorzystane mechanizmy:** Łączenie tabel (`JOIN`), zaawansowane podzapytanie korelacyjne z operatorem `NOT EXISTS`, filtrowanie po dacie.

SQL

```
CREATE VIEW Klienci_Do_Aktywizacji AS
SELECT k.Imie AS Imie_Klienta,
       k.Nazwisko AS Nazwisko_Klienta,
       k.Adres_Email,
       t.Nazwa AS Posiadany_Karnet
FROM Klient k
JOIN Karnet kar ON k.Id_Klienta = kar.Id_Klienta
JOIN Typ_Karnetu t USING (Id_Typu_Karnetu)
WHERE kar.Data_Zakonczenia >= CURDATE()
AND NOT EXISTS (
    SELECT 1
    FROM Rezerwacje r
    WHERE r.Id_Klienta = k.Id_Klienta
);
```



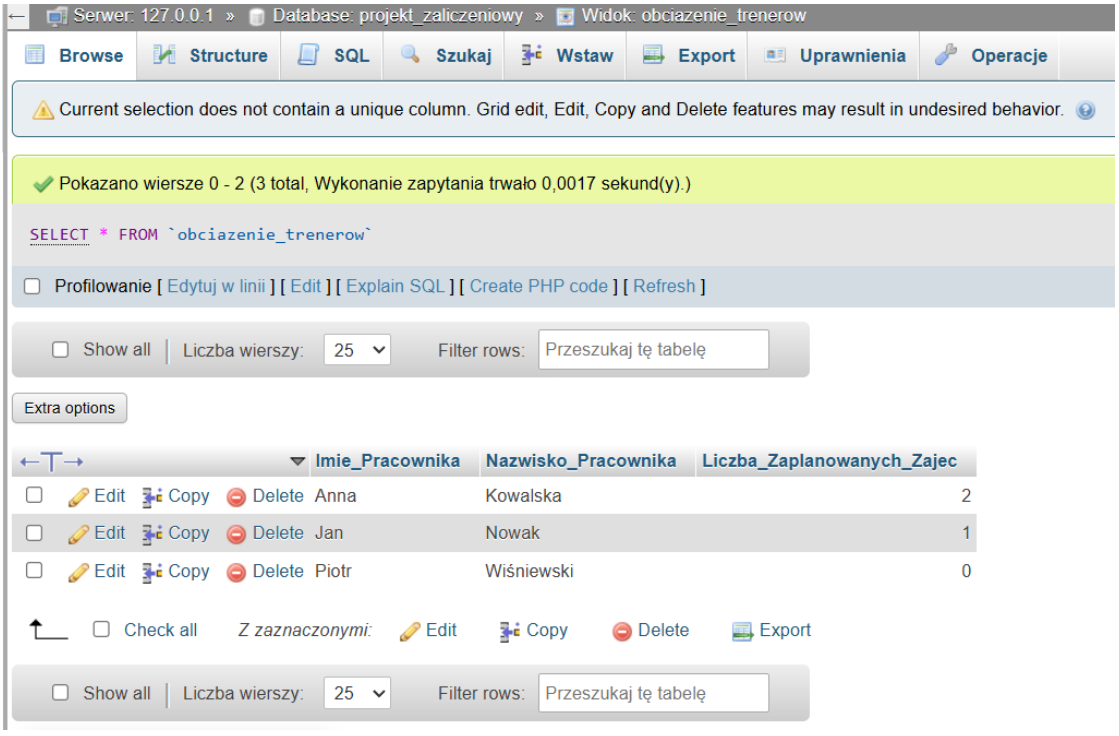
Raport 5: Aktywność pracowników

- **Praktyczne zastosowanie:** Analiza obciążenia trenerów – zlicza, ile sesji zajęciowych zostało przydzielonych każdemu pracownikowi.

- **Wykorzystane mechanizmy:** Podzapytanie jako wirtualna tabela w klauzuli FROM (FROM (SELECT ...)), zagnieżdżone łączenia tabel, aliasowanie.

SQL

```
CREATE VIEW Obciazenie_Trenerow AS
SELECT
    Zestawienie.Imie_Pracownika,
    Zestawienie.Nazwisko_Pracownika,
    Zestawienie.Liczba_Zaplanowanych_Zajec
FROM (
    SELECT
        p.Imie AS Imie_Pracownika,
        p.Nazwisko AS Nazwisko_Pracownika,
        COUNT(zg.Id_Zajec) AS Liczba_Zaplanowanych_Zajec
    FROM Pracownik p
    LEFT JOIN Zajecia_Grupowe zg ON p.Id_Pracownika =
zg.Id_Pracownika
    GROUP BY p.Id_Pracownika, p.Imie, p.Nazwisko
) AS Zestawienie
ORDER BY Zestawienie.Liczba_Zaplanowanych_Zajec DESC;
```

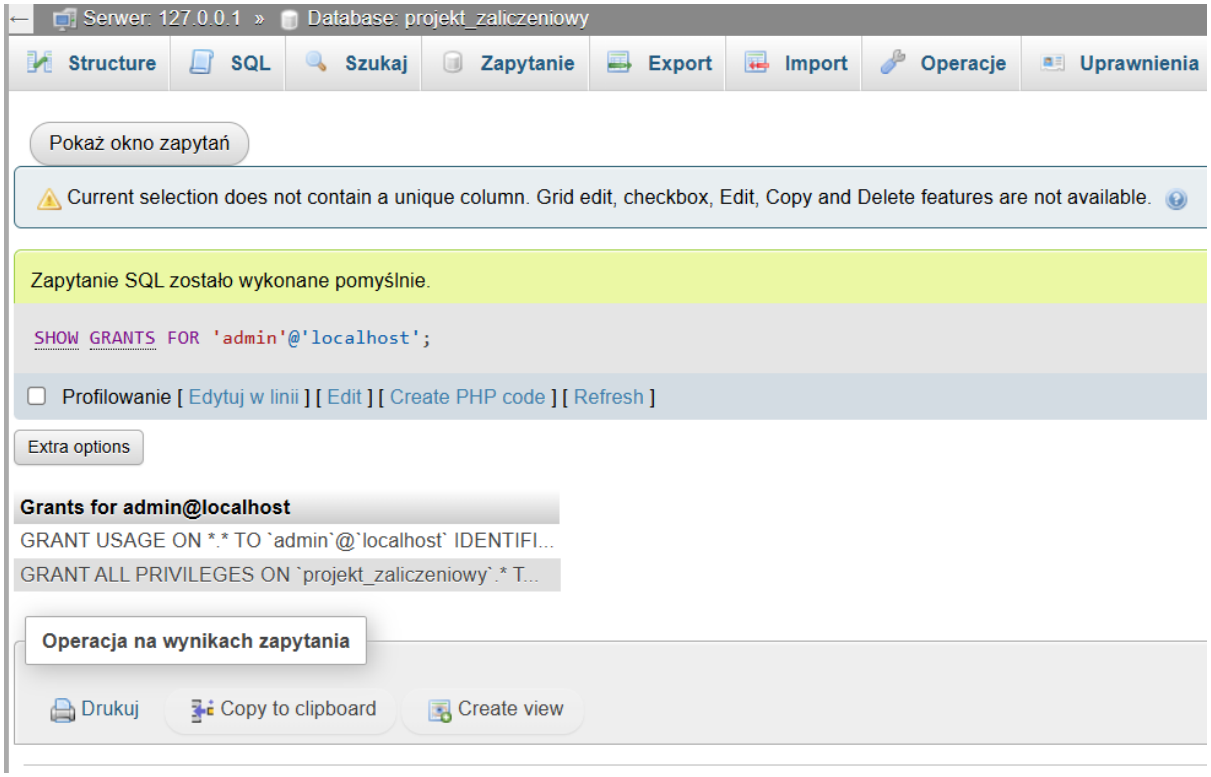


6. Dodaj sekcję Użytkownicy i uprawnienia:

W celu zapewnienia bezpieczeństwa danych w systemie oraz ochrony danych osobowych klientów, wdrożono mechanizm kontroli dostępu oparty na rolach (Role-Based Access Control). Utworzono trzy dedykowane konta użytkowników, a przydzielone im uprawnienia odzwierciedlają ich rzeczywiste obowiązki w klubie fitness, zgodnie z zasadą najmniejszego uprzywilejowania.

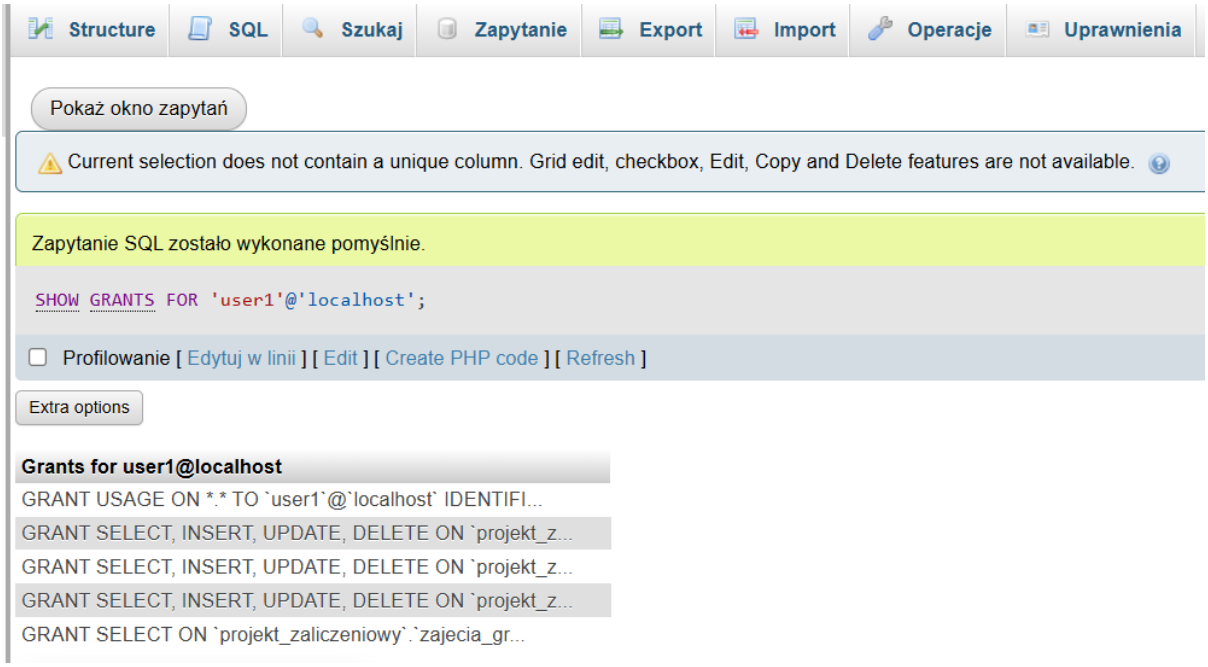
Konta użytkowników oraz ich uprawnienia

1. **Konto: `admin` (Administrator Bazy Danych)**
    a. **Uprawnienia:** `ALL PRIVILEGES` na wszystkie tabele.
    b. **Uzasadnienie:** Główny administrator systemu. Posiada pełny dostęp do struktury bazy oraz wszystkich danych. Wykorzystuje to konto wyłącznie do celów konfiguracyjnych, tworzenia kopii zapasowych oraz zarządzania systemem.

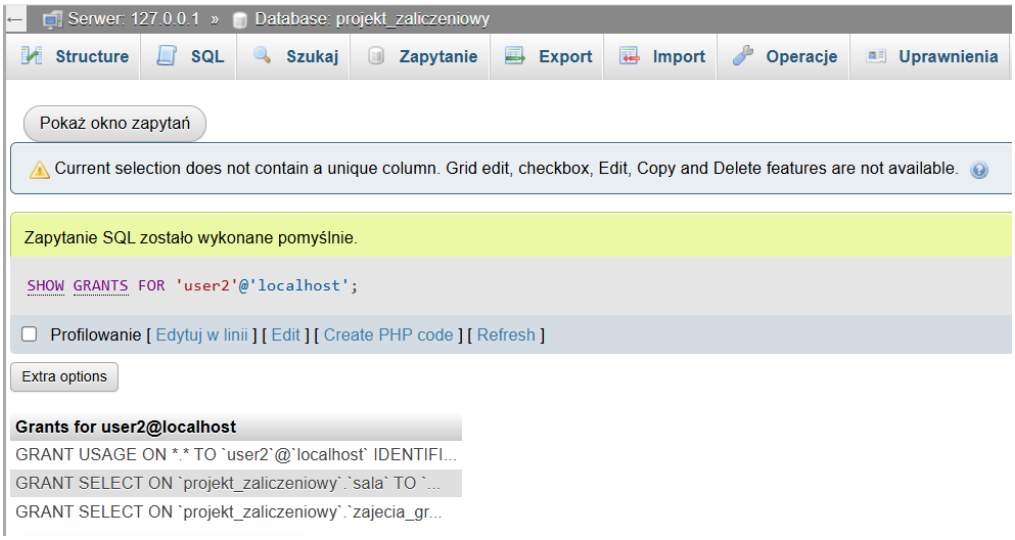


2. **Konto: `user1` (Pracownik Recepcji)**
    a. **Uprawnienia operacyjne:** `SELECT`, `INSERT`, `UPDATE`, `DELETE` dla tabel: `Klient`, `Karnet`, `Rezerwacje`.
    b. **Uprawnienia do odczytu:** `SELECT` dla tabeli `Zajecia_Grupowe`.
    c. **Uzasadnienie:** Recepcjonista to osoba pracująca z systemem na co dzień. Musi mieć możliwość dodawania nowych klientów, edytowania ich danych, sprzedaży karnetów oraz zarządzania rezerwacjami. Jednocześnie dostęp do grafiku zajęć grupowych został ograniczony

wyłącznie do odczytu (SELECT) – pracownik widzi plan, aby informować klientów, ale nie może go samodzielnie modyfikować ani usuwać (leży to w gestii menedżera).



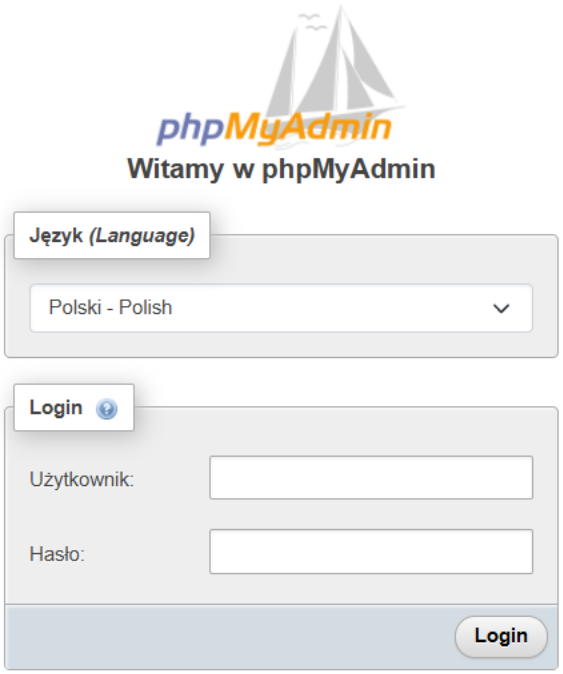
3. **Konto: user2 (Trener / Instruktor)**
    a. **Uprawnienia:** SELECT dla tabel: `Zajecia_Grupowe`, `Sala`.
    b. **Uzasadnienie:** Konto o mocno ograniczonych uprawnieniach. Trener loguje się do systemu wyłącznie w celu sprawdzenia swojego grafiku pracy (kiedy prowadzi zajęcia i w jakiej sali). Nie posiada dostępu do wrażliwych danych osobowych klientów, historii rezerwacji ani danych finansowych (karnety).



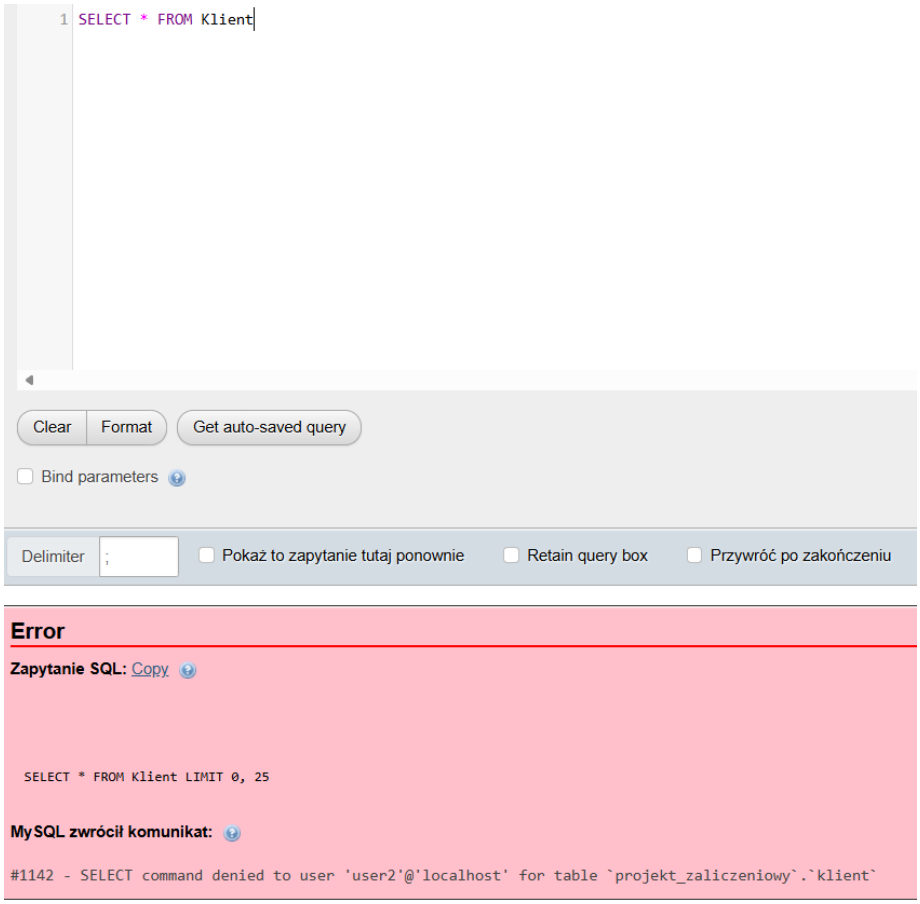
Testy bezpieczeństwa, logowanie i limity dostępu

W celu weryfikacji poprawności wdrożonych zabezpieczeń, przetestowano mechanizm logowania oraz nałożone restrykcje na konta poszczególnych pracowników.

1. Ekran logowania do systemu Dostęp do bazy został odpowiednio zabezpieczony. Każdy użytkownik przed rozpoczęciem pracy musi przejść proces autoryzacji, podając swój unikalny login i hasło.



2. Weryfikacja limitów – konto Trenera (user2) Zalogowano się na konto trenera i podjęto celową próbę nieautoryzowanego odczytu danych osobowych klientów (zapytanie `SELECT * FROM Klient`). Zgodnie z nadanymi uprawnieniami (tylko do odczytu sal i grafiku), system poprawnie zablokował operację, zwracając błąd braku dostępu. Stanowi to dowód na poprawne działanie limitów i ochronę danych wrażliwych.



3. Możliwości pracy z danymi – konto Recepcji (user1) Po przelogowaniu na konto pracownika recepcji, wykonano identyczne zapytanie. System autoryzował operację i poprawnie zwrócił listę klientów, co udowadnia, że to konto posiada odpowiednie możliwości pracy i edycji danych niezbędnych do codziennej obsługi klubu.

Pokazano wiersze 0 - 8 (9 total, Wykonanie zapytania trwało 0,0002 sekund(y).)

```
SELECT * FROM Klient;
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Liczba wierszy: 25 Filter rows: Przeszukaj tę tabelę Sortuj wg klucza: Żaden

Extra options

	Id_Klienta	Imie	Nazwisko	Adres_Email	Numer_Telefonu	Id_Adresu
Edit Copy Delete	1	Marta	Zielińska	marta@email.com	111222333	1
Edit Copy Delete	2	Krzysztof	Lewandowski	krzysztof@email.com	222333444	2
Edit Copy Delete	3	Zofia	Wójcik	zofia@email.com	333444555	3
Edit Copy Delete	4	Tomasz	Kaczmarek	tomasz.k@email.com	444555666	4
Edit Copy Delete	5	Agnieszka	Szymańska	aga.szym@email.com	555666777	5
Edit Copy Delete	6	Michał	Dąbrowski	michal.d@email.com	666777888	6
Edit Copy Delete	7	Katarzyna	Kozłowska	kasia.k@email.com	777888999	7
Edit Copy Delete	8	Piotr	Jankowski	piotr.j@email.com	888999000	8
Edit Copy Delete	9	Olaf	Nowak	olaf@email.com	100200300	1

7. Transakcje i współbieżność

W celu zachowania pełnej spójności danych (zgodnie z właściwościami ACID), w systemie zaimplementowano mechanizm transakcji. Wykorzystano instrukcje `START TRANSACTION`, `COMMIT` (zatwierdzenie) oraz `ROLLBACK` (wycofanie), co zapobiega powstawaniu niekompletnych lub błędnych modyfikacji w przypadku awarii.

7.1. Scenariusz 1: Pomyślna transakcja (COMMIT)

Opis: Klient kupuje nowy karnet i przy okazji prosi o aktualizację swojego numeru telefonu w systemie. Obie operacje (sprzedaż i aktualizacja danych) muszą zostać potraktowane jako jedna spójna całość.

Pokaż okno zapytań

MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0003 sekund(y).)

```
START TRANSACTION;
```

[Edytuj w linii] [Edit] [Create PHP code]

1 row inserted.
Id wstawionego wiersza: 12 (Wykonanie zapytania trwało 0,0004 sekund(y).)

```
INSERT INTO Karnet (Id_Klienta, Id_Typu_Karnetu, Data_Rozpoczecia, Data_Zakonczenia) VALUES (1, 3, '2026-05-19', '2027-05-19');
```

[Edytuj w linii] [Edit] [Create PHP code]

1 row affected. (Wykonanie zapytania trwało 0,0003 sekund(y).)

```
UPDATE Klient SET Numer_Telefonu = '999888777' WHERE Id_Klienta = 1;
```

[Edytuj w linii] [Edit] [Create PHP code]

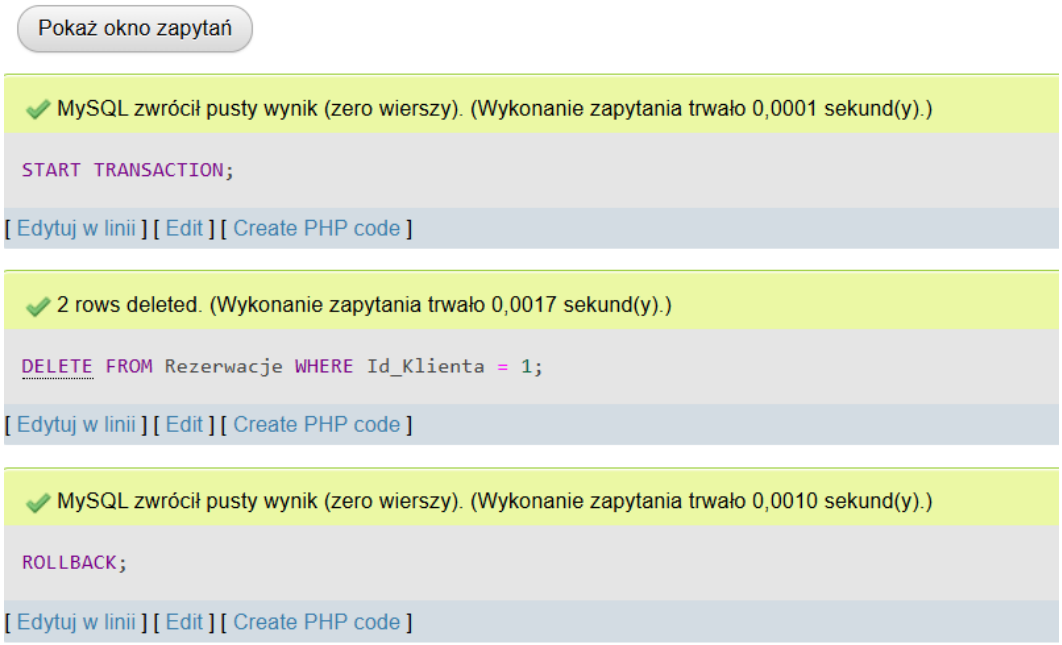
MySQL zwrócił pusty wynik (zero wierszy). (Wykonanie zapytania trwało 0,0019 sekund(y).)

```
COMMIT;
```

[Edytuj w linii] [Edit] [Create PHP code]

7.2. Scenariusz 2: Anulowanie transakcji z powodu pomyłki (ROLLBACK)

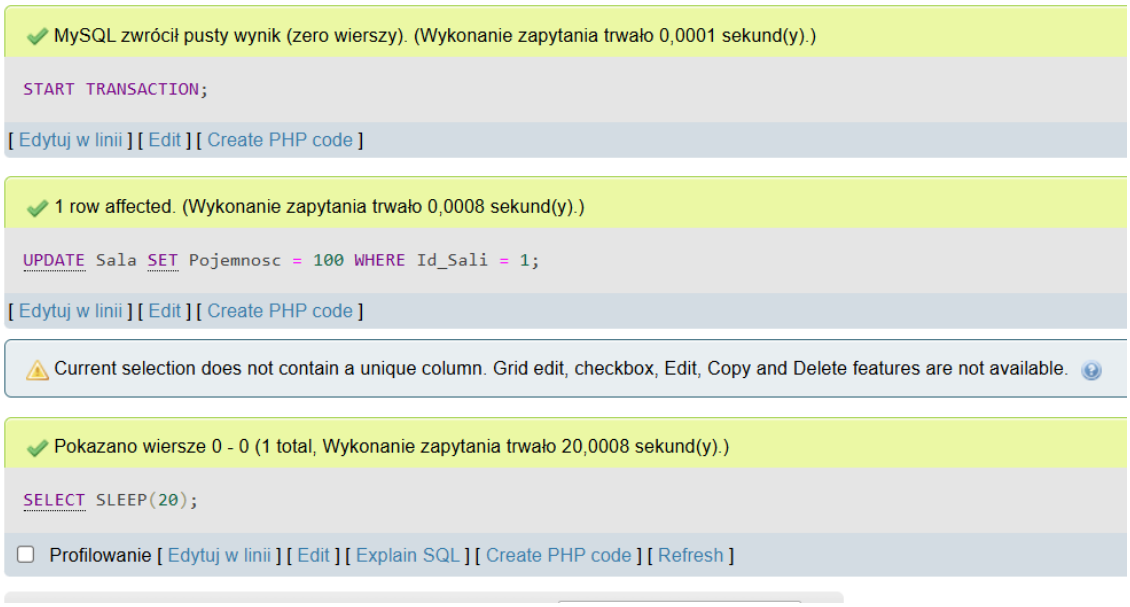
Opis: Recepcjonista otrzymuje prośbę o usunięcie błędnej rezerwacji. Z powodu pomyłki w kodzie próbuje usunąć wszystkie rezerwacje wybranego klienta. Orientuje się w błędzie przed zatwierdzeniem zmian i wycofuje całą transakcję, ratując dane.



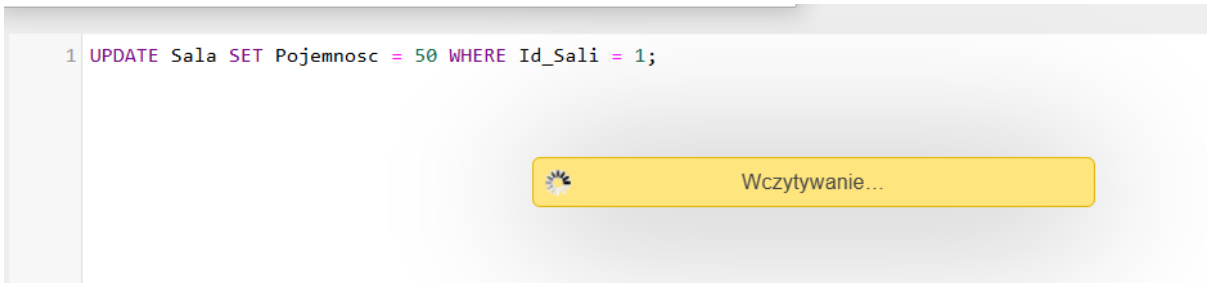
7.3. Scenariusz 3: Testowanie współbieżności (Konflikt dostępu i blokady)

Opis: Ostatnim testem była symulacja sytuacji, w której dwóch użytkowników (dwie osobne sesje/karty) próbuje w tym samym ułamku sekundy zmodyfikować te same dane – w tym przypadku pojemność Sali nr 1. Celem testu było udowodnienie, że baza danych potrafi zarządzać współbieżnością i zapobiega nadpisywaniu się danych.

Przebieg testu i wynik: W pierwszej sesji rozpoczęto transakcję (`START TRANSACTION`) i zaktualizowano rekord, a następnie celowo "uśpiono" bazę na 20 sekund (`SELECT SLEEP(20);`), wstrzymując komendę `COMMIT`. System relacyjny założył w tym momencie blokadę na edytowany wiersz.



Gdy w drugiej sesji podjęto próbę zmiany tego samego rekordu (`UPDATE Sala SET Pojemnosc = 50...`), baza danych zachowała się w 100% poprawnie. Zamiast wykonać zapytanie od razu, zablokowała je, co widać na poniższym zrzucie ekranu jako "Wczytywanie...".



Zapytanie z drugiej sesji musiało czekać w kolejce, aż pierwsza transakcja zostanie zatwierdzona lub wycofana. Udowadnia to całkowitą spójność bazy danych oraz odporność na konflikty edycji.

8. Podsumowanie i wnioski końcowe

Zaprojektowana baza danych realizuje założenia biznesowe dla klubu fitness (uwagi – w README, „Do poprawy"). Główne wnioski z realizacji projektu to:

1. **Optymalna struktura:** Zastosowanie 3. postaci normalnej (3NF) i prawidłowych relacji wyeliminowało redundancję danych, co ułatwi ewentualną rozbudowę systemu.

2. **Automatyzacja raportów:** Użycie widoków pozwala menedżerom na natychmiastowy dostęp do kluczowych statystyk, takich jak sprzedaż czy obłożenie zajęć.
3. **Bezpieczeństwo dostępu:** Wdrożenie ról użytkowników o ściśle określonych uprawnieniach (np. recepcja, trener) skutecznie chroni wrażliwe dane klientów przed nieautoryzowanym odczytem.
4. **Odporność na błędy:** Zaimplementowane mechanizmy transakcji oraz blokad zapobiegają utracie spójności danych przy jednoczesnej pracy wielu użytkowników.